6-3 · Producer / Consumer 애플리케이션 · 실습 2.5H

Producer / Consumer 코드와 주문 이벤트 파이프라인

파이썬으로 양쪽을 만들고, 유실·중복·LAG을 직접

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

2.5시간 뒤에는

① confluent-kafka 로 주문 이벤트 30건을 발행하고 delivery report 를 본다

② flush 를 빼고 1000건을 보내 유실을 재현한다

③ poll 루프 Consumer 로 고객별 누적 금액을 집계한다

④ 이어읽기·처음부터 읽기·수동 커밋·LAG 을 각각 확인한다

실습점검에서 compose 를 켜고 orders 토픽(파티션 3)을 비워 둔 뒤 시작합니다

실습 흐름

일곱 단계 — ~/data-eng/week6 한 폴더에서 끝까지

호스트에서 붙으므로 bootstrap.servers 는 localhost:9092 — 6-2 의 EXTERNAL 리스너가 여기서 쓰입니다

1. 환경 준비	2. Producer	3. 일부러 실패	4·5. Consumer	6·7. 커밋·LAG
.venv 활성화 confluent-kafka	주문 30건 키 = 고객	flush 없이 1000건	누적 집계 이어읽기	수동 커밋 LAG 관찰

1단계

환경 준비 — 버전이 찍히면 준비 완료

위치	cd ~/data-eng/week6
kafka 확인	docker compose ps
가상환경	source .venv/bin/activate
설치	pip install confluent-kafka
확인	python -c "import confluent_kafka; print(confluent_kafka.version())"

이번 주 Python 클라이언트는 confluent-kafka 로 통일 — 버전은 2.16.1 처럼 찍힌다(실측)

2단계

producer.py 핵심 줄 — 주문 이벤트를 1초에 하나씩

줄	뜻
Producer({"bootstrap.servers": ...})	localhost:9092 로 연결
def on_delivery(err, msg)	건별 성공·실패 콜백
order = {order_id, customer, amount}	JSON 주문 이벤트
p.produce("orders", key=customer, ...)	고객 ID 를 키로 — 같은 고객은 같은 칸
p.poll(0)	쌓인 콜백 처리
finally: p.flush()	버퍼를 비우고 종료

python producer.py — 30건이 1초 간격으로 나간다

확인: 「전송 완료 topic=orders partition=N offset=M」 30줄 — 출력에 고객이 없으니 칸은 4단계 consumer 의 [p칸:오프셋] 으로

2단계

같은 키라도 CLI 와 파이썬은 칸이 다르다

6-2 CLI (자바 클라이언트)

한 클라이언트 안에서는 같은 키 = 언제나 같은 칸

- user1 → 0번 (6-2 실측)
- 자바의 기본 해시
- 둘을 섞어 보내면 같은 키가 두 칸으로 갈라진다

VS

오늘 파이썬 (librdkafka)

기본 해시가 자바와 다르다

- user1 → 1번 · user2 · user3 → 2번
- 0번은 빈 칸 (실측)
- "partitioner": "murmur2_random" 이면 같아짐

3단계 — 일부러 실패시키기

flush 를 빼고 1000건 — 이론의 경고를 직접

sleep 을 없애고 flush 도 뺀 producer_noflush.py 로 orders-test 토픽에 보냅니다

토픽 생성	producer_noflush	sleep 3	오프셋 조회	flush 넣고 재실행
orders-test 파티션 1	1000건 produce flush 없음	잠시 기다린 뒤	get-offsets 로 쌓인 개수 확인	파일 끝에 p.flush() 다시 조회

3단계 — 일부러 실패시키기

1000 건 보냈다고 찍었는데 — 실제로 쌓인 것은

flush() 없음

버퍼에 다 있다가 통째로 사라진다

- orders-test:0:0 — 한 건도 없다 (실측, 세 번 모두)
- %4|...|TERMINATE| ... 1000 messages ... use flush()
- 오류가 아니라 경고 — 정상 종료로 보인다

VS

flush() 있음

버퍼를 다 보낼 때까지 기다린 뒤 끝난다

- echo 'p.flush()' >> producer_noflush.py
- orders-test:0:1000 — 경고 없이 정확히
- 끝나는 곳마다 — try/finally 안에서 flush()

4단계

consumer.py 핵심 줄 — poll 루프와 고객별 누적

줄	뜻
"group.id": "order-agg"	이 Consumer 가 속한 그룹
"auto.offset.reset": "earliest"	새 그룹은 처음부터
c.subscribe(["orders"])	토픽 구독
msg = c.poll(1.0)	없으면 1초 기다렸다 None
if msg.error(): ...	오류를 조용히 넘기지 않는다
json.loads(msg.value().decode("utf-8"))	JSON 역직렬화
finally: c.close()	그룹 정상 탈퇴 + 오프셋 정리

터미널 2 가 새 창이면 cd ~/data-eng/week6 && source .venv/bin/activate 부터 · JSONDecodeError 로 멈추면 실습점검 3단계로 orders 비우기

5·6·7단계

이어읽기 · 처음부터 · 수동 커밋 · LAG

1. **같은 그룹으로 다시 켜면 — 이어읽기**
   오프셋이 커밋되어 있어 이미 집계한 메시지를 다시 읽지 않는다

2. **group.id 를 order-agg-2 로 — 처음부터**
   earliest 가 새 그룹에만 적용된다는 이론을 확인

3. **enable.auto.commit False + c.commit(msg)**
   5의 배수 금액이면 예외 — 커밋하지 않고 break, 다시 켜면 그 메시지부터

LAG 관찰

꺼 두면 쌓이고

켜면 0

> **--describe --group order-agg · 꺼 둔 동안 실측 LAG 1번 4 · 2번 26 → 켜면 0 · 0번 줄이 없는 것은 메시지가 없는 칸**

6단계 — 왜 break 로 멈추나

커밋은 「이 파티션의 여기까지」라는 뜻

처음 판 — 건너뜀(continue)

같은 파티션 뒤 메시지를 commit(msg) 하면

- 「여기까지」가 실패한 메시지 뒤로 넘어간다
- 다시 켜도 실패한 것을 읽지 않는다
- 실측 — 30 건 중 7 건 실패, 다시 켜니 0 건

VS

지금 판 — 멈춤(break)

실패하면 커밋하지 않고 끝낸다

- 처리 실패: 13 에서 멈춘다
- 다시 켜도 13 에서 또 멈춘다 (실측)
- 유실 없음 · 같은 것을 또 읽음 — 실무는 재시도 뒤 DLQ

확인 포인트

이 2.5시간이 끝날 때 확인할 것

1. 「전송 완료 …」 30줄 — 칸은 1번 · 2번뿐, user1 은 1번
2. flush 없이 보낸 orders-test 는 0 건, flush 를 넣으면 1000 건
3. Consumer 터미널에 고객별 누적 금액이 늘어난다
4. 같은 그룹은 새 메시지만, 다른 그룹은 전부 다시 읽는다
5. 수동 커밋은 실패에서 멈추고, 다시 켜도 같은 메시지에서 멈춘다
6. Consumer 를 끄면 LAG 이 쌓이고, 켜면 전 파티션 0 이 된다

미니 과제: consumer.py 둘 동시 실행 · acks all 추가 · 집계를 파일이나 SQLite 에 저장

6-3 · 실습 2.5H 마무리

다음은 정리·회고 (0.5H)

코드 리뷰 표로 빠뜨리기 쉬운 여섯 가지를 점검하고, 6-4 CDC 에 필요한 MySQL 준비를 확인합니다

- ✓ **코드 리뷰 — flush·poll·close·키·오류·멱등**
- ✓ **오늘의 체크리스트**
- ✓ **자주 겪는 오류 네 가지**
- ✓ **6-4 를 위한 준비 확인**

가져갈 것

producer.py · consumer.py · 집계 출력